

A FIELD GUIDE · CHAPTER 0 — WELCOME

유한한 세계에서 피어나는 완벽한 산수

갈루아 필드(GF)에서 출발해 리드-솔로몬(Reed-Solomon) 부호와 LFSR(선형 피드백 시프트 레지스터)까지 — QR코드 한 장, USB 한 개, 인공위성의 한 신호 안에 숨겨진 유한체의 수학을 따라가 봅니다.

대상

고등학생 · 수학 호기심이 있는 누구나

분량

7개 장 · 실행 가능한 Python 예제 14개

준비물

Python 3.10+ (외부 라이브러리 불필요)

01	왜 "유한한 산수"가 필요할까?	· 모듈러 연산과 만나기
02	갈루아 필드 $GF(p)$	· 소수 위에서 자라는 산수
03	갈루아 필드 $GF(2^n)$ — 비트의 우주	· 다항식, XOR, 그리고 마법
04	LFSR — 선형 피드백 시프트 레지스터	· 비트가 춤추는 메트로놈
05	리드-솔로몬 부호 — 망가져도 살아남는 데이터	· QR코드와 CD 안의 마법
06	실전: QR코드처럼 만들어 보는 미니 RS	· 직접 만들고, 망가뜨리고, 복구하기
07	활용 사례 모음	· 어디에 쓰이는지, 왜 쓰이는지

CHAPTER 01

왜 "유한한 산수"가 필요할까?

우리가 학교에서 배우는 산수는 무한히 큰 수를 다룹니다. 하지만 컴퓨터, 위성, QR코드 속 세상은 — 모두 유한합니다. 그 유한한 세계에서도 "더하기·곱하기·나누기"가 깔끔하게 돌아가게 하려면, 새로운 종류의 산수가 필요합니다.

§ 시계 산수 — 우리가 이미 알고 있는 유한한 산수

아침 9시에 일을 시작해 8시간을 일하면 몇 시일까요? 답은 17시이지만, 12시간 시계로는 **5시**입니다. 시계는 12에 도달하면 다시 0(혹은 12)으로 돌아갑니다. 이 "돌아옴"이 바로 모듈러 산수(modular arithmetic)의 핵심입니다.

 비유

시계 비유: 12시간 시계는 0, 1, 2, ..., 11 이 열두 개의 숫자만 가지고 있는 작은 세계입니다. 이 안에서도 우리는 자유롭게 더하고 뺄 수 있습니다.

예: $9 + 8 = 17 \equiv 5 \pmod{12}$ — "17은 12로 나눈 나머지가 5".

이렇게 "어떤 수로 나눈 나머지"만 가지고 산수하는 세계를, 우리는 $\mathbb{Z}/n\mathbb{Z}$ 또는 그냥 **mod n** 이라고 부릅니다.

§ 그런데 — 나눗셈은 항상 잘 될까?

더하기·빼기·곱하기는 mod n 안에서 항상 잘 됩니다. 하지만 **나눗셈**은 까다롭습니다. 예를 들어 mod 6 안에서 " $2 \div 4$ "는 답을 가질까요?

나눗셈은 본질적으로 "곱셈의 역원을 곱하는 것"입니다. 그러니까 $2 \div 4$ 는 2×4^{-1} . 그런데 mod 6 에서 4의 역원이 존재할까요? $4 \times x \equiv 1 \pmod{6}$ 을 만족하는 x 가 있는지 표로 확인해 봅시다:

x	0	1	2	3	4	5
$4 \cdot x \pmod{6}$	0	4	2	0	4	2

4의 곱셈 결과: 1이 한 번도 나오지 않는다 → 4는 mod 6 에서 역원이 없다

△ 핵심 통찰

mod n 의 세계에서, 모든 0이 아닌 수가 역원을 가지려면 n이 소수(prime)여야 합니다. n이 합성수이면 일부 수들은 나눗셈에 실패합니다. 그래서 우리는 소수를 사랑하게 됩니다.

§ "필드(Field)"란 무엇인가

수학자들은 "+, -, ×, ÷ (단, ÷는 0이 아닌 수에 대해)"가 모두 잘 정의되고, 결합법칙·교환법칙·분배법칙이 성립하는 집합을 **체(體, field)**라고 부릅니다. 유리수 $\mathbb{Q}$, 실수 $\mathbb{R}$, 복소수 $\mathbb{C}$ 가 우리에게 익숙한 무한 필드입니다.

그러면 **유한한 원소 개수**만 가진 필드도 존재할까요? 네, 존재합니다. 그것을 발견하고 체계화한 19세기 프랑스 청년 수학자의 이름을 따서 갈루아 필드(Galois Field)라고 부릅니다. 원소 개수가 q 개인 갈루아 필드를 $GF(q)$ 또는 $\mathbb{F}_q$ 로 씁니다.

"수학은 패턴을 찾고, 패턴을 통해 우주의 비밀에 다가가는 일이다."

— 갈루아의 작업을 기리며

📖 짧은 인물 스케치 — 에바리스트 갈루아 (1811–1832)

갈루아는 20세에 결투에서 사망했지만, 그 전날 밤 친구에게 보낸 편지에 평생의 수학적 발견이 담겨 있었습니다. 그가 만든 군론(group theory)과 유한체 이론은 100년 뒤 디지털 통신·암호학·오류 정정 부호의 토대가 됩니다.

§ 💻 첫 번째 실행 가능한 예제 — mod p 에서 역원 찾기

이제 직접 Python으로 확인해 봅시다. 외부 라이브러리는 사용하지 않습니다.

```
# =====
# 예제 1.1 - 모듈러 산수와 곱셈 역원 (multiplicative inverse)
# =====
# 목표: mod p (p는 소수) 에서 a의 곱셈 역원을 찾는다.
#       즉, a * x ≡ 1 (mod p) 을 만족하는 x를 구한다.
#
# 두 가지 방법을 비교한다:
#   (A) 무식한 탐색 (브루트포스)
#   (B) 페르마의 소정리(Fermat's Little Theorem): a^(p-2) ≡ a^(-1)
#
# 페르마의 소정리: p가 소수이고 gcd(a,p)=1 이면 a^(p-1) ≡ 1 (mod p)
#                  양변에 a^(-1) 을 곱하면 a^(p-2) ≡ a^(-1)
# =====

def inverse_brute(a: int, p: int) -> int:
    """방법 A - 0..p-1 을 차례로 시험. O(p) 시간."""
```

PYTHON

```

a %= p
for x in range(1, p):
    if (a * x) % p == 1:
        return x
raise ValueError(f"{a}의 mod {p} 역원이 존재하지 않음 (p가 소수가 아닐 수 있음)")

def inverse_fermat(a: int, p: int) -> int:
    """방법 B - 페르마의 소정리. 파이썬 내장 pow(a, p-2, p) 사용. O(log p)."""
    # pow(a, e, m) 은 a^e mod m 을 O(log e) 에 계산한다 (제곱-곱 알고리즘).
    return pow(a, p - 2, p)

# ----- 검증 -----
if __name__ == "__main__":
    p = 13 # 소수
    print(f"=== mod {p} 에서의 곱셈 역원 표 ===")
    print(f"{'a':>3} | {'brute':>5} | {'fermat':>6} | a*inv mod p")
    print("-" * 38)
    for a in range(1, p):
        b = inverse_brute(a, p)
        f = inverse_fermat(a, p)
        check = (a * f) % p
        print(f"{a:>3} | {b:>5} | {f:>6} | {check}")

# 실행 결과 (발체):
#   a | brute | fermat | a*inv mod p
#   1 |      1 |      1 | 1
#   2 |      7 |      7 | 1   ← 2 * 7 = 14 ≡ 1 (mod 13)
#   3 |      9 |      9 | 1
#   ...
# 모든 a에 대해 역원이 존재! p가 소수이기 때문이다.

```

✅ 이 장에서 배운 것

① 모듈러 산수는 "유한한 시계" 같은 세계다. ② 나눗셈이 항상 가능하려면 모듈러스가 소수여야 한다. ③ 페르마의 소정리는 역원을 $O(\log p)$ 에 빠르게 구하게 해 준다. ④ 이렇게 사칙연산이 완벽히 닫힌 유한 세계가 바로 $\text{GF}(p)$.

CHAPTER 02

갈루아 필드 $\text{GF}(p)$ — 소수 위에서 자라는 산수

가장 단순한 형태의 갈루아 필드, $\text{GF}(p)$. 여기서 p 는 소수입니다. 원소는 $\{0, 1, 2, \dots, p-1\}$ — 끝. 산수는 모두 $\text{mod } p$. 정말 작은 우주에서, 우리는 무엇을 할 수 있을까요?

§ $\text{GF}(5)$ 의 덧셈·곱셈표를 직접 보자

5는 소수이므로 $\text{GF}(5) = \{0, 1, 2, 3, 4\}$ 는 멋진 필드입니다. 표를 만들어 보면:

덧셈표 (a + b mod 5)

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

곱셈표 (a × b mod 5)

×	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

곱셈표의 0이 아닌 행/열을 보면 — **각 행이 0을 제외한 모든 원소를 정확히 한 번씩 포함**합니다. 이것이 바로 "역원이 존재한다"는 사실의 시각적 증거입니다. 예: 두 번째 행 (1, 2, 3, 4) 어디에든 1이 나오므로, 곱해서 1이 되는 짝이 반드시 있다.

§ 원시근 (Primitive Root) — 하나의 수로 모두 표현하기

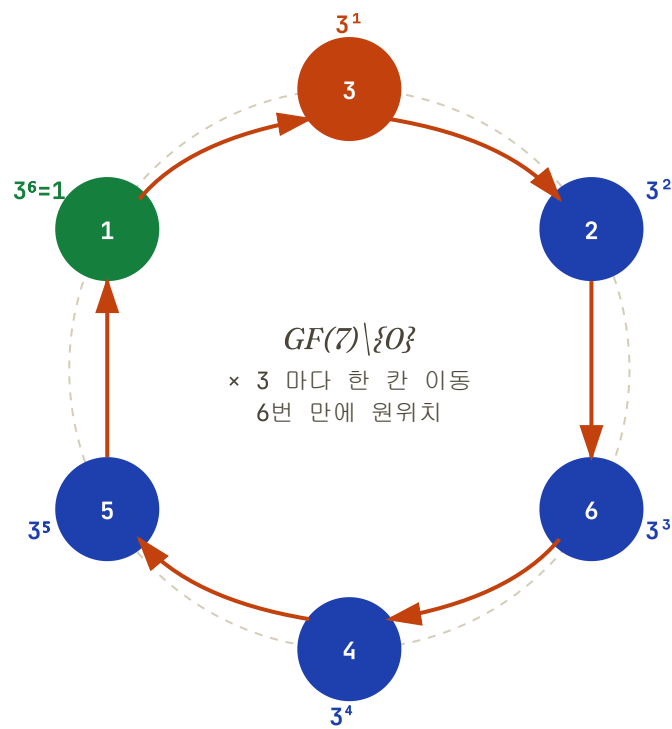
위 곱셈표의 "2의 행"을 보면: 2, 4, 1, 3 — 즉 $2^1 = 2, 2^2 = 4, 2^3 = 3, 2^4 = 1 \pmod{5}$. 한 수 2를 거듭제곱하는 것만으로 GF(5)의 0이 아닌 모든 원소가 등장합니다! 이런 수를 원시근(primitive root) 또는 **생성원(generator)**이라고 부릅니다.

💡 왜 중요한가

원시근이 있다는 것은 **"필드를 한 줄로 펼칠 수 있다"**는 뜻입니다. 이걸 나중에 LFSR이 "최대 주기"를 갖게 되는 비밀이고, 디지털 통신과 암호의 토대입니다. GF(p)에서는 항상 원시근이 존재합니다 (가우스가 증명).

§ 시각적으로 보는 GF(7)의 곱셈

아래 다이어그램은 GF(7)의 원시근 3을 거듭제곱했을 때 어떻게 모든 원소가 한 번씩 나타나는지 보여 줍니다.



$GF(7)$ 에서 3은 원시근. 거듭제곱이 6개 원소(1~6)를 모두 한 번씩 휘감고 돌아온다.

§ 🖥️ GF(p) 클래스 구현

PYTHON

```
# =====
# 예제 2.1 - GF(p) 산술 클래스
# =====
# 갈루아 필드 GF(p) 의 원소를 표현하는 클래스를 만든다.
# 연산자 오버로딩으로 a + b, a * b, a / b 가 자연스럽게 동작하도록.
# =====

class GFp:
    """GF(p) - 소수 p에 대한 유한체의 원소."""
    p = 7 # 클래스 변수: 어떤 GF인지

    __slots__ = ("v",) # 메모리 절약 + 속성 오타 방지

    def __init__(self, v: int):
        # 항상 [0, p) 범위로 정규화
        self.v = v % self.p

    # ----- 사칙연산 -----
    def __add__(self, o): return GFp(self.v + self._raw(o))
    def __sub__(self, o): return GFp(self.v - self._raw(o))
    def __mul__(self, o): return GFp(self.v * self._raw(o))
    def __truediv__(self, o):
        # 페르마의 소정리: o^(p-2) ≡ o^(-1)
        inv = pow(self._raw(o), self.p - 2, self.p)
        return GFp(self.v * inv)
    def __pow__(self, n: int): return GFp(pow(self.v, n, self.p))
    def __neg__(self): return GFp(-self.v)

    # ----- 비교/출력 -----
    def __eq__(self, o): return self.v == self._raw(o)
    def __hash__(self): return hash((self.p, self.v))
    def __repr__(self): return f"GF({self.p})({self.v})"

    @classmethod
    def _raw(cls, o):
        """상대값이 GFp 인스턴스든 정수든 정수 v 로 반환."""
        return o.v if isinstance(o, GFp) else o % cls.p

# ----- 원시근 찾기 (보너스) -----
```

```
def find_primitive_root(p: int) -> int:
    """GF(p)의 가장 작은 원시근 g 찾기. g의 차수가 정확히 p-1 이어야 함."""
    # p-1 의 소인수를 모두 구한 뒤, 각 소인수 q 에 대해 g^((p-1)/q) ≠ 1 이면 OK
    n = p - 1
    factors = set()
    m = n
    f = 2
    while f * f <= m:
        while m % f == 0:
            factors.add(f); m //= f
        f += 1
    if m > 1: factors.add(m)
    for g in range(2, p):
        if all(pow(g, n // q, p) != 1 for q in factors):
            return g
    raise RuntimeError("원시근을 못 찾음 (p가 소수가 아닐 수도)")

# ----- 시연 -----
if __name__ == "__main__":
    # GF(7) 에서 놀기
    GFp.p = 7
    a, b = GFp(3), GFp(5)
    print(f"a = {a}, b = {b}")
    print(f"a + b = {a + b}")      # 3+5=8≡1
    print(f"a - b = {a - b}")      # 3-5=-2≡5
    print(f"a * b = {a * b}")      # 15≡1
    print(f"a / b = {a / b}")      # 3 * 5-1 = 3 * 3 = 9 ≡ 2
    print(f"a^4 = {a ** 4}")       # 81 ≡ 4
    print(f"원시근: {find_primitive_root(7)}") # 3

# 출력:
# a = GF(7)(3), b = GF(7)(5)
# a + b = GF(7)(1)
# a - b = GF(7)(5)
# a * b = GF(7)(1)
# a / b = GF(7)(2)
# a^4 = GF(7)(4)
# 원시근: 3
```

 언어 비교

C/C++: 매크로/템플릿으로 GF 산술을 쓰면 `typedef uint8_t gf_t` 같은 8비트 타입 위에 인라인 함수로 구현 — 캐시 친화적, 임베디드에서 표준.

Go: 제네릭이 1.18+ 에 추가되어 GF 구현 가능. 하지만 산술 연산자 오버로딩이 없어 가독성이 떨어진다.

Python: `__add__` 등으로 자연스러운 수식형 표현 가능. 학습/검증용에 최적.

CHAPTER 03

갈루아 필드 GF(2ⁿ) — 비트의 우주

컴퓨터는 비트로 말합니다. 0과1. 그런데 GF(2) — 즉 *mod 2* — 만 가지고는 너무 답답합니다. 원소가 2개뿐이니깐요. 어떻게 하면 8비트(256가지) 나 16비트(65,536가

지) 짜리 필드를 만들 수 있을까요? 답은 — 다항식을 사용하는 것입니다.

§ GF(2)의 모든 것 — XOR 산수

먼저 가장 작은 갈루아 필드 $GF(2) = \{0, 1\}$ 부터:

+	0	1
0	0	1
1	1	0

×	0	1
0	0	0
1	0	1

🔗 핵심 발견

$GF(2)$ 의 덧셈은 곧 **XOR**입니다! 그리고 곱셈은 **AND**입니다. 즉, $GF(2)$ 산술은 컴퓨터에서 가장 빠른 비트 연산으로 자동 구현됩니다. 이게 $GF(2)$ 의 친척들이 디지털 회로의 본부에 있는 이유입니다.

§ 왜 그냥 mod 4, mod 8 을 쓰면 안 되나?

"비트 4개 → 16가지 → mod 16 쓰자!" — 솔깃하지만, 16은 소수가 아니라서 **필드가 아닙니다**. 예를 들어 mod 16에서 $2 \times 8 = 16 \equiv 0$. 0이 아닌 두 수를 곱했는데 결과가 0! 이건 필드의 규칙을 위반합니다.

그래서 수학자들은 더 영리한 방법을 씁니다. **다항식으로 원소를 표현**하고, 어떤 특별한 다항식으로 나눈 나머지로 산수를 하는 것입니다.

§ 다항식으로 표현하는 $GF(2^3) = GF(8)$

$GF(2^3)$ 의 원소는 비트 3개로 된 모든 수, 즉 8가지입니다: 000, 001, 010, ..., 111. 이걸 다음처럼 **다항식**으로 봅니다 (계수는 $GF(2)$ — 즉 0 또는 1):

$$101 \Leftrightarrow x^2 + 1 \quad 110 \Leftrightarrow x^2 + x \quad 011 \Leftrightarrow x + 1 \quad 111 \Leftrightarrow x^2 + x + 1$$

다항식끼리 더하는 건 단순한 XOR이지만, **곱하면 차수가 커집니다**. 예: $(x + 1)(x^2 + 1) = x^3 + x^2 + x + 1$ — 차수가 3이 됐고, 더 이상 3비트로 표현이 안 됩니다. 여기서 등장하는 것이 기약 다항식(irreducible polynomial)입니다.

기약 다항식 — "다항식 세계의 소수"

기약 다항식은 $GF(2)$ 위에서 인수분해되지 않는 다항식입니다. $GF(2^3)$ 을 만들 때 우리는 차수 3짜리 기약 다항식 하나를 고릅니다 — 가장 흔한 선택은:

$$p(x) = x^3 + x + 1$$

그러면 $GF(2^3)$ 의 모든 곱셈은 다음과 같이 합니다:

1

두 다항식을 곱한다

예: $(x^2 + 1)(x + 1) = x^3 + x^2 + x + 1$ (계수는 GF(2) – XOR로 더함)

2

기약 다항식 $p(x)$ 로 나눈 나머지를 취한다

$x^3 + x^2 + x + 1 \div (x^3 + x + 1) =$ 몫 1, 나머지 x^2 – 즉 결과는 x^2 , 비트로는 100

3

이게 답이다!

$(x^2 + 1)(x + 1) \equiv x^2 \pmod{x^3 + x + 1}$, 비트로 표현: $101 \times 011 = 100$

 비유

비유: GF(p) 가 "시계 산수"라면, GF(2ⁿ)은 "다항식 시계"입니다. 차수가 너무 커지면 기약 다항식으로 나눠서 작게 줄입니다 – 시계 바늘이 12를 넘으면 다시 0으로 가듯이.

§ GF(2³)의 모든 원소를 한 번에 – 원시 다항식의 마법

$x^3 + x + 1$ 은 단지 기약일 뿐 아니라 **원시(primitive)**입니다. 즉, x 라는 원소를 거듭제곱하면 0을 제외한 GF(8)의 모든 원소가 한 번씩 나타납니다. 이 x 를 보통 α 라고 부르며, GF(2ⁿ)의 모든 원소를 α 의 거듭제곱으로 적을 수 있습니다:

거듭제곱	다항식 형태	비트	10진수
0	0	000	0
α^0	1	001	1
α^1	x	010	2
α^2	x^2	100	4
α^3	$x+1$ ($\because \alpha^3=\alpha+1$)	011	3
α^4	x^2+x	110	6
α^5	x^2+x+1	111	7
α^6	x^2+1	101	5
α^7	1 (= α^0 , 한 바퀴!)	001	1

α 의 거듭제곱이 GF(8)의 0이 아닌 7개 원소를 모두 한 번씩 휘감는다.

§ GF(2⁸) 구현 – AES, QR코드, CD가 쓰는 그것

이제 진짜 실용적인 필드, GF(2⁸) 를 만들어 봅시다. 원소가 256개라서 한 바이트로 표현되며, AES 암호와 QR코드, 리드-솔로몬 부호의 표준입니다. 표준 기약 다항식은 $p(x) = x^8 + x^4 + x^3 + x + 1$, 비트로는 0x11B .

```
# =====
# 예제 3.1 – GF(2^8) 구현 (AES/QR/리드솔로몬에서 쓰는 표준)
# =====
# 기약 다항식: x^8 + x^4 + x^3 + x + 1   (= 0b1_0001_1011 = 0x11B)
# 모든 원소는 0..255 의 정수로 표현 (각 비트가 다항식 계수).
# =====

PRIM = 0x11B   # 기약 다항식 (9비트)

def gf_add(a: int, b: int) -> int:
    """GF(2^n) 덧셈 = XOR.   덧셈과 뺄셈은 같다."""
    return a ^ b

def gf_mul_slow(a: int, b: int) -> int:
    """
    GF(2^8) 곱셈 – 교과서 알고리즘 (peasant multiplication).
    a, b: 0..255 정수.   반환: 0..255.
    """
    result = 0
    # b의 각 비트를 보면서 a를 적절히 시프트해 더한다.
    for i in range(8):
        if b & 1:                # b의 최하위 비트가 1이면
            result ^= a          #   현재 a를 결과에 XOR로 더함
        b >>= 1
        # a 를 한 비트 왼쪽으로 – 즉 x를 곱한 효과
        carry = a & 0x80         # 최상위 비트가 다음 단계에서 차수 8을 만들지 검사
        a <<= 1
        if carry:                # 차수 8이 됐으면 기약 다항식으로 환원
            a ^= PRIM            # PRIM의 9번째 비트(차수 8)와 상쇄되어 8비트로 줄어듦
        a &= 0xFF                # 8비트로 마스크
    return result

# ----- 검증 -----
if __name__ == "__main__":
    print(f"0x53 * 0xCA = 0x{gf_mul_slow(0x53, 0xCA):02X}   (AES 문서 예: 0x01)")
    # AES 명세서에 나오는 유명 예제: 0x53 * 0xCA = 0x01

# 실행 결과:
#   0x53 * 0xCA = 0x01
```

⚡ 성능 최적화 – 로그/지수 테이블

위 곱셈은 비트마다 루프를 도므로 느립니다. 실전에서는 **로그 테이블**을 미리 만듭니다: 원시근 α 를 정해서 모든 원소 $v = \alpha^k$ 로 표현한 뒤, $a \times b = \alpha^{\log a + \log b}$ 처럼 덧셈으로 바꿉니다 (지수의 mod 255). 이러면 한 번의 곱셈이 단지 3번의 테이블 lookup으로 끝납니다.

```
# =====
# 예제 3.2 – GF(2^8) 빠른 구현 (로그/지수 테이블)
# =====
# 원시원소  $\alpha = 0x03$  (= x+1) 을 거듭제곱하면 GF(256)의 0이 아닌 원소가
# 모두 한 번씩 등장한다 (주기 255).
#
# gf_exp[i] =  $\alpha^i \bmod p(x)$       ← 지수 → 값
# gf_log[v] = i such that  $\alpha^i=v$  ← 값 → 지수
#
# 한 번 만들면 곱셈/나눗셈/거듭제곱이 모두 O(1).
# =====

PRIM    = 0x11B
GEN     = 0x03   # 원시 원소  $\alpha$  (= x + 1)
FIELD   = 256
```



```

gf_exp = [0] * (2 * FIELD)    # 두 배 크기로 만들어 mod 연산을 없앤다
gf_log = [0] * FIELD

def _build_tables():
    """프로그램 시작 시 한 번만 호출."""
    x = 1
    for i in range(FIELD - 1):    # 0..254
        gf_exp[i] = x
        gf_log[x] = i
        # x = x * GEN (gf_mul_slow를 인라인)
        x ^= ((x << 1) ^ (PRIM if x & 0x80 else 0)) & 0xFF # x *= α(=x+1) 의 한 방법
        # 더 명확한 방법:
    # 위 한 줄 코드는 가독성이 떨어지니, 아래처럼 다시 쓴다:

def _build_tables_clear():
    """가독성 좋은 버전."""
    x = 1
    for i in range(FIELD - 1):
        gf_exp[i] = x
        gf_log[x] = i
        # x = x * 0x03 (= x*(x+1)) 을 실제 GF 곱셈으로 계산
        x = gf_mul_slow(x, GEN)
    # 두 배 영역 채우기 (덧셈에서 mod 안 쓰려고)
    for i in range(FIELD - 1, 2 * (FIELD - 1)):
        gf_exp[i] = gf_exp[i - (FIELD - 1)]

_build_tables_clear()

def gf_mul(a: int, b: int) -> int:
    """O(1) 곱셈."""
    if a == 0 or b == 0: return 0
    return gf_exp[gf_log[a] + gf_log[b]]    # mod 안 함! exp 테이블이 2배 길어서 OK

def gf_div(a: int, b: int) -> int:
    """O(1) 나눗셈."""
    if b == 0: raise ZeroDivisionError
    if a == 0: return 0
    # 로그 영역에서 빼고 mod 255
    return gf_exp[(gf_log[a] - gf_log[b]) % (FIELD - 1)]

def gf_pow(a: int, n: int) -> int:
    """O(1) 거듭제곱."""
    if a == 0: return 0
    return gf_exp[(gf_log[a] * n) % (FIELD - 1)]

def gf_inv(a: int) -> int:
    """곱셈 역원: α(-k) = α(255-k)."""
    if a == 0: raise ZeroDivisionError
    return gf_exp[(FIELD - 1) - gf_log[a]]

# ----- 검증 -----
if __name__ == "__main__":
    # AES 문서 예제 다시 확인
    print(f"빠른 곱셈: 0x53 * 0xCA = 0x{gf_mul(0x53, 0xCA):02X}")
    # 곱셈 역원 확인
    a = 0x53
    inv = gf_inv(a)
    print(f"0x53의 역원: 0x{inv:02X},    0x53 * 0x{inv:02X} = 0x{gf_mul(a, inv):02X}")

# 실행 결과:
#   빠른 곱셈: 0x53 * 0xCA = 0x01
#   0x53의 역원: 0xCA,    0x53 * 0xCA = 0x01    ← AES S-Box의 핵심!

```

AES 암호의 S-Box는 본질적으로 "0x11B로 정의된 GF(256)에서의 곱셈 역원"입니다. 즉 우리가 방금 만든 함수가 AES의 심장을 이미 구현한 셈입니다.

§ 두 구현 방식 비교

방법	곱셈 1회 비용	메모리	장단점
peasant (반복)	~16 비트 연산	0 KB	메모리 없는 임베디드, 사이드채널 저항
log/exp 테이블	3 테이블 lookup	~768 B	가장 빠름, 가장 흔함
전체 곱셈표	1 lookup	64 KB	최고 속도, 캐시 압박
CLMUL 명령어 (x86)	1 사이클	0 KB	최신 CPU 한정, AES-NI 동반

CHAPTER 04

LFSR — 선형 피드백 시프트 레지스터

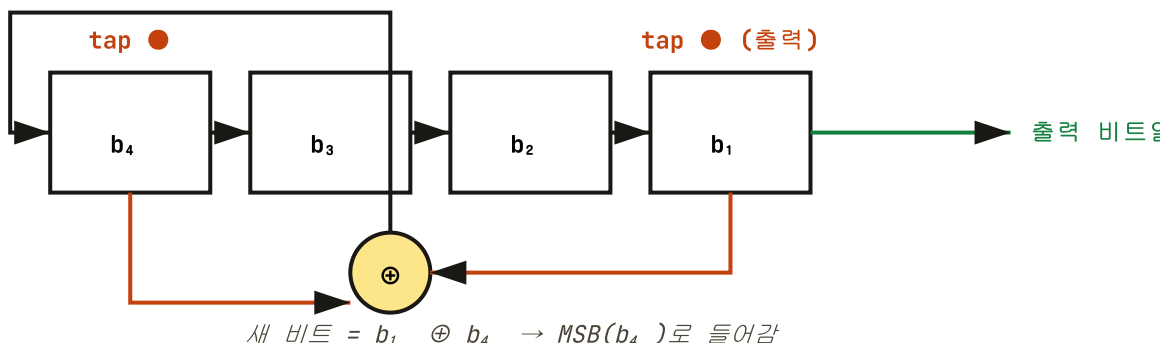
이제까지의 갈루아 필드 이론을 회로 한 조각으로 응축한 것이 바로 *LFSR*입니다. 레지스터의 비트들이 시계처럼 짹짹거리고, *XOR* 게이트가 새로운 비트를 만들어 다시 집어넣습니다. 이 단순한 장치가 — 의외로 — 위성 *GPS*, *Bluetooth*, *MP3* 플레이어, 그리고 옛 게임의 효과음까지 들어가 있습니다.

§ 한 줄로 말하면

*LFSR*이란 $GF(2)$ 위의 다항식 곱셈 회로다. 그저, 그뿐이다.

§ 4비트 LFSR의 해부도

아래 그림은 가장 유명한 4비트 LFSR입니다. 비트 4개의 레지스터, 그리고 **비트 1과 비트 4**를 XOR한 결과를 다시 비트 4에 넣습니다. 이것 계속 반복하면? 0이 아닌 모든 4비트 패턴(15가지)이 정확히 한 번씩 나타난 뒤 다시 처음으로 돌아옵니다.



§ 다항식과 회로의 일대일 대응

탭 위치를 다항식으로 적으면:

$$\text{탭이 1, 4 위치} \Leftrightarrow p(x) = x^4 + x + 1$$

이 다항식이 GF(2) 위에서 **원시 다항식**이면, LFSR은 최대 주기 $2^n - 1$ 를 갖습니다 — 그래서 0이 아닌 모든 상태를 거치며 반복됩니다.

§ 4비트 LFSR — 한 사이클 추적

초기값 **0001** 부터 시작해 한 클럭씩 진행합니다 (상태 표기: $b_4b_3b_2b_1$).

클럭	b_4 b_3 b_2 b_1	10진 수	출력 비트 (b_1)
0	0 0 0 1	1	1
1	1 0 0 0	8	0
2	1 1 0 0	12	0
3	1 1 1 0	14	0
4	1 1 1 1	15	1
5	0 1 1 1	7	1
6	1 0 1 1	11	1
7	0 1 0 1	5	1
8	1 0 1 0	10	0
9	1 1 0 1	13	1
...	...	...	...
15	0 0 0 1	1 (원 위치!)	1

15 클럭 만에 정확히 한 바퀴. 4비트 LFSR의 최대 주기 = $2^4 - 1 = 15$.

§ 🖥️ 실행 가능한 LFSR — Fibonacci 형과 Galois 형

LFSR에는 두 가지 구현 방식이 있습니다. 둘은 수학적으로 동등하지만 회로가 다릅니다:

```
# =====
# 예제 4.1 - Fibonacci LFSR & Galois LFSR
# =====
# n비트 LFSR을 두 가지 방식으로 구현하고 같은 비트열이 나오는지 확인.
#
# Fibonacci 형: 여러 탭의 XOR을 입력 쪽에 모은다 (위 다이어그램).
# Galois 형   : 출력 비트가 1이면 전체 상태에 다항식 마스크를 XOR.
#               하드웨어 효율이 더 좋다.
# =====

def fibonacci_lfsr(seed: int, taps: list[int], n_bits: int, steps: int) -> list[int]:
    """
    seed      : 초기 상태 (0이 아닌 정수)
```

PYTHON

```

taps    : 탭 위치들 (1-indexed, 예: [3,4]).   곧 다항식  $x^4+x^3+1$ .
n_bits  : 레지스터 길이
steps   : 추출할 비트 수
반환    : 출력 비트열 (각 클럭마다 LSB 가 빠져나옴)
"""

state = seed
out = []
for _ in range(steps):
    out.append(state & 1)                # LSB가 출력
    # 탭들 XOR 계산
    fb = 0
    for t in taps:
        fb ^= (state >> (t - 1)) & 1    # 1-indexed → 0-indexed
    state >>= 1                          # 시프트
    state |= fb << (n_bits - 1)          # 새 비트를 MSB에 삽입
return out

def galois_lfsr(seed: int, poly_mask: int, steps: int) -> list[int]:
    """
    poly_mask: 원시 다항식의 비트 표현 (최고차항 제외).
               예:  $x^4+x^3+1 \rightarrow 0b1001 = 9$  ( $x^4$  빼고  $x^3$ ,  $x^0$  만 OR)
               단, Galois 형에서는 탭 위치가 Fibonacci 와 '반대'이다:
               즉 같은 다항식을 쓰려면 비트를 reverse 해야 한다.
    """
    state = seed
    out = []
    for _ in range(steps):
        lsb = state & 1
        out.append(lsb)
        state >>= 1
        if lsb:
            state ^= poly_mask            # 출력비트가 1이면 다항식을 XOR
    return out

# ----- 비교 검증 -----
if __name__ == "__main__":
    # 동일한 다항식  $x^4 + x + 1$  을 두 방식으로
    fib = fibonacci_lfsr(seed=0b0001, taps=[1, 4], n_bits=4, steps=20)
    # Galois 형: 다항식 마스크 0b1001 (=  $x^3 + 1$  부분, 최고차  $x^4$  빼고)
    gal = galois_lfsr(seed=0b1001, poly_mask=0b1001, steps=20)

    print("Fibonacci 출력:", ''.join(map(str, fib)))
    print("Galois      출력:", ''.join(map(str, gal)))

    # 주기 길이 측정
    def find_period(seed, taps, n):
        state = seed
        seen = {state: 0}
        i = 0
        while True:
            fb = 0
            for t in taps: fb ^= (state >> (t - 1)) & 1
            state = (state >> 1) | (fb << (n - 1))
            i += 1
            if state in seen:
                return i - seen[state]
            seen[state] = i

    p = find_period(1, [1, 4], 4)
    print(f"4비트 LFSR 주기: {p}   (최대 = {2**4 - 1})")

# 실행 결과:
#   Fibonacci 출력: 10001111010110010001   ← 주기 15

```

Galois 출력: 11101011001000111101 ← 시퀀스는 다르지만 같은 통계적 성질
4비트 LFSR 주기: 15 (최대 = 15) ← 풀 주기!

§ 🎯 LFSR이 만드는 비트열의 신비한 성질

최대 주기 LFSR이 만드는 비트열은 **M-sequence(maximal-length sequence)**라고 부르며, 진짜 난수처럼 보이는 통계적 성질을 갖습니다:

균형성

0과 1의 개수가 거의 같음

한 주기 안에 1은 2^{n-1} 개, 0은 $2^{n-1} - 1$ 개. 거의 반반.

실행 길이

연속된 같은 비트 분포가 예측 가능

길이 1짜리 run이 절반, 길이 2짜리 1/4, ... — 진짜 동전 던지기와 비슷.

자기상관

스스로와의 상관관계가 거의 0

한 칸 시프트하면 상관계수가 $-1/(2^n - 1)$. 거의 무상관.

하지만!

암호용으로 안전하지 않음

$2n$ 개 비트만 보면 다항식을 역으로 알아낼 수 있음 (Berlekamp-Massey). 그래서 암호에서는 LFSR을 비선형 함수와 결합해서 씁니다.

§ 🎮 보너스: 8비트 LFSR로 만드는 옛 게임 노이즈

닌텐도 NES의 노이즈 채널은 사실 15비트 LFSR이었습니다. 8비트로 직접 만들어 봅시다:

```
# =====
# 예제 4.2 - 8비트 LFSR로 NES 풍 노이즈 비트열 만들기
# =====
# 다항식: x^8 + x^6 + x^5 + x^4 + 1    (탭 [4,5,6,8])    -- 원시 다항식
# 주기: 255

def nes_noise(seed: int = 0x01, length: int = 255) -> list[int]:
    """길이 length 의 LFSR 노이즈 비트열을 반환."""
    state = seed & 0xFF
    out = []
    for _ in range(length):
        out.append(state & 1)
        # 탭 XOR
        fb = ((state >> 0) ^ (state >> 2) ^ (state >> 3) ^ (state >> 4)) & 1
        # Fibonacci 형: 시프트 후 새 비트는 MSB에
        # 위 식은 다항식과 일대일 대응되도록 비트 위치를 잘 고른 것
        state = (state >> 1) | (fb << 7)
    return out

if __name__ == "__main__":
    bits = nes_noise()
    # 처음 64비트만 출력
    print("노이즈 비트열:", ''.join(map(str, bits[:64])))
```

PYTHON

```
print(f"1의 개수: {sum(bits)} / 255 (이상적: 128)")

# 주기 확인
seed = 0x01
state = seed
for i in range(1, 1000):
    fb = ((state >> 0) ^ (state >> 2) ^ (state >> 3) ^ (state >> 4)) & 1
    state = (state >> 1) | (fb << 7)
    if state == seed:
        print(f"주기 = {i}")
        break

# 실행 결과:
#   노이즈 비트열: 1000000010110011100000010111110100110001001011000010001110100110
#   1의 개수: 128 / 255
#   주기 = 255    ← 최대 주기!
```

🎵 응용 한 줄

각 비트를 +1 또는 -1로 해석해 오디오 샘플로 흘려보내면, 우리에게 익숙한 "치~익" 하는 슈퍼마리오의 발사 효과음, 인베이터의 폭발음이 됩니다. 한 줄짜리 XOR 회로가 한 시대의 사운드를 만든 셈입니다.

CHAPTER 05

리드-솔로몬 부호 — 망가져도 살아남는 데이터

QR코드를 가위로 자르거나, CD에 흠집이 나거나, 화성 탐사선이 우주 잡음 때문에 비트를 잃어도, 데이터는 멀쩡합니다. 그 비밀의 절반은 1960년 어빙 리드(Irving Reed)와 거스 솔로몬(Gus Solomon)이 만든 부호에 있습니다 — 그리고 그 모든 마법은 갈루아 필드 위에서 일어납니다.

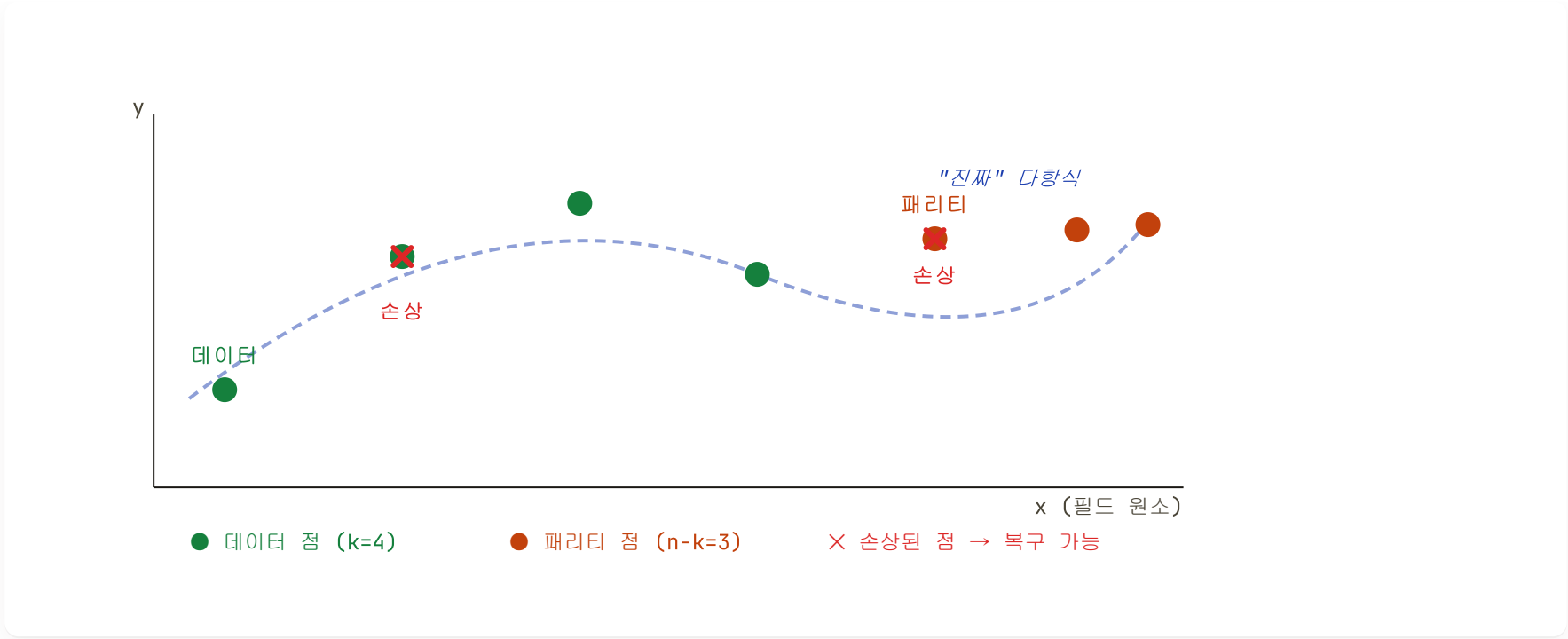
§ 핵심 아이디어 — "다항식은 점으로 결정된다"

🔗 비유

비유 — 두 점이면 직선 하나: 두 점만 알면 직선이 유일하게 정해집니다. 세 점이면 2차 곡선이 정해지고, k 개의 점이면 $k - 1$ 차 다항식이 유일하게 정해집니다.

이걸 거꾸로 이용합니다: 우리가 전송하려는 데이터가 k 개의 수라면, 이걸 $k - 1$ 차 다항식의 "계수"로 보고, n 개 ($n > k$) 점에서의 함숫값을 계산해 전송합니다. 그러면 받는 쪽은 그 중 k 개만 살아남으면 다항식을 복원할 수 있고, 그 다항식의 계수가 곧 데이터입니다!

§ 그림으로 보는 리드-솔로몬의 정신



k 개의 데이터를 $k - 1$ 차 다항식의 계수로 보고 n 개 점에서 평가. 손상된 점이 $(n - k)/2$ 개 이하면 원본 다항식이 유일하게 복원된다.

§ 오류 정정 능력 — 공식 한 줄

n 개 점 중 k 개가 데이터, $(n - k)$ 개가 패리티일 때:

이레이저(위치 알려진 오류) 최대 $n - k$ 개 | 일반 오류 최대 $\lfloor (n - k)/2 \rfloor$ 개 정정

예시 — QR코드 L 등급

QR코드 버전 1, 오류정정 레벨 L 은 데이터 19바이트 + 패리티 7바이트 = 총 26바이트. 따라서 최대 3바이트의 오류를 자동 복구합니다. 더 높은 레벨인 H 는 패리티가 17바이트라서 8바이트 오류까지 정정 가능 — 그래서 QR코드에 로고를 가운데 박아도 읽힙니다!

§ 인코딩 알고리즘 — 단계별

리드-솔로몬은 두 가지 등가 방식이 있습니다. 표준은 "생성 다항식을 이용한 **systematic 인코딩**"입니다 — 원본 데이터가 그대로 보존되고, 끝에 패리티 바이트가 붙는 방식. CD/QR/DVD가 모두 이 방식을 씁니다.

1

생성 다항식 $g(x)$ 만들기

$g(x) = (x - \alpha^0)(x - \alpha^1) \cdots (x - \alpha^{2t-1})$ — 여기서 $t = (n - k)/2$ 가 정정 가능 오류 수. 모든 계수는 $\text{GF}(2^8)$.

2

데이터를 다항식 $m(x)$ 로 본다

예: 데이터가 $[D_0, D_1, D_2]$ 라면 $m(x) = D_0x^2 + D_1x + D_2$.

3

$m(x) \cdot x^{n-k}$ 를 $g(x)$ 로 나눠 나머지 $r(x)$ 를 구한다

다항식 나눗셈을 GF(2⁸)에서 수행한다. 이 나머지의 계수들이 곧 패리티 바이트가 된다.

4

전송: 데이터 + 패리티

최종 부호어 $c(x) = m(x) \cdot x^{n-k} + r(x)$. 데이터 바이트는 손대지 않고 뒤에 패리티만 붙으니 "systematic"이다.

§ 🌀 디코딩 알고리즘 — 망가진 부호어 복구

1

증후군(Syndrome) 계산

수신한 부호어 $r(x)$ 에 $\alpha^0, \alpha^1, \dots, \alpha^{2t-1}$ 을 대입. 모두 0이면 오류 없음, 하나라도 0이 아니면 오류 발견.

2

오류 위치 다항식 $\Lambda(x)$ 찾기

증후군으로부터 오류 위치를 알려주는 다항식을 만든다. 대표 알고리즘: **Berlekamp–Massey** 또는 **Peterson–Gorenstein–Zierler**.

3

Chien Search — 오류 위치 확정

$\Lambda(x)$ 의 근을 찾는다. 그 근의 역수가 오류가 발생한 정확한 위치.

4

Forney 알고리즘 — 오류 값 계산

오류 위치에서의 정확한 오류 값(어떻게 비트가 뒤집혔는지)을 구한다. 부호어에서 XOR로 빼면 끝!

§ 🖥️ 다항식 산술 (GF(2⁸) 위에서)

리드-솔로몬을 만들기 전에, GF(2⁸) 위에서 다항식을 다루는 도구가 필요합니다.

```
# =====
# 예제 5.1 - GF(2^8) 위의 다항식 산술
# =====
# 다항식을 list 로 표현. [a, b, c] = a*x^2 + b*x + c (계수가 최고차부터)
# 모든 계수는 0..255 (GF(256) 원소).
# =====

# 이전 장에서 만든 gf_add/gf_mul/gf_div/gf_pow/gf_log/gf_exp 를 사용한다고 가정.
```

PYTHON


```
# 여기선 핵심 다항식 연산만:
```

```
def poly_add(p, q):
    """두 다항식을 더한다 (XOR)."""
    r = [0] * max(len(p), len(q))
    for i, x in enumerate(p): r[i + len(r) - len(p)] = x
    for i, x in enumerate(q): r[i + len(r) - len(q)] ^= x
    return r
```

```
def poly_scale(p, x):
    """다항식의 모든 계수에 스칼라 x 를 곱한다."""
    return [gf_mul(c, x) for c in p]
```

```
def poly_mul(p, q):
    """다항식 곱셈."""
    r = [0] * (len(p) + len(q) - 1)
    for j, qj in enumerate(q):
        for i, pi in enumerate(p):
            r[i + j] ^= gf_mul(pi, qj)
    return r
```

```
def poly_eval(p, x):
    """호너의 방법으로 p(x) 평가. O(deg) 시간."""
    # p[0]*x^(n-1) + p[1]*x^(n-2) + ... + p[n-1]
    # = ((p[0]*x + p[1])*x + p[2])*x + ... + p[n-1]
    y = p[0]
    for c in p[1:]:
        y = gf_mul(y, x) ^ c
    return y
```

```
def poly_div(dividend, divisor):
    """다항식 긴 나눗셈. 반환: (몫, 나머지)."""
    out = list(dividend)
    n = len(divisor)
    for i in range(len(dividend) - n + 1):
        coef = out[i]
        if coef != 0:
            # 한 항씩 빼나간다
            for j in range(1, n):
                if divisor[j] != 0:
                    out[i + j] ^= gf_mul(divisor[j], coef)
    # 마지막 (n-1) 개 항이 나머지
    sep = -(n - 1)
    return out[:sep], out[sep:]
```

§ 본격 리드-솔로몬 인코더 / 디코더

```
# =====
# 예제 5.2 – Reed-Solomon 인코더 (Systematic)
# =====
# n: 부호어 길이, k: 데이터 길이, n-k: 패리티 길이 (= 2*t)
# 모든 산술은 GF(2^8), 기약 다항식 0x11B.
# =====

def rs_generator_poly(nsym):
    """
    패리티 수 nsym 에 대한 생성 다항식:
        g(x) = (x - α^0)(x - α^1) ... (x - α^(nsym-1))
    GF(2) 위에선 (x - α^i) = (x XOR α^i) 이므로 그냥 더한다.
    """
    g = [1]
    for i in range(nsym):
        # (x + α^i) = [1, gf_exp[i]]
```

PYTHON

```

        g = poly_mul(g, [1, gf_exp[i]])
    return g

def rs_encode(msg_in, nsym):
    """
    systematic 인코딩:
        c(x) = m(x) * x^nsym - (m(x) * x^nsym mod g(x))
    ↑ 즉, 데이터 뒤에 nsym 개의 0을 붙인 뒤,
        그 다항식을 g(x)로 나눈 '나머지'로 마지막 nsym 개 자리를 채운다.
    이렇게 하면 원본 데이터가 결과의 앞부분에 그대로 보존된다.
    """
    gen = rs_generator_poly(nsym)
    # msg + [0] * nsym
    msg_out = list(msg_in) + [0] * nsym
    # 다항식 나눗셈을 손으로 (in-place)
    for i in range(len(msg_in)):
        coef = msg_out[i]
        if coef != 0:
            for j in range(1, len(gen)):
                msg_out[i + j] ^= gf_mul(gen[j], coef)
    # 나눗셈 후 앞부분에는 원본 데이터가 그대로, 뒷부분에 패리티가 들어 있음
    msg_out[:len(msg_in)] = msg_in
    return msg_out

# ----- 시연 -----
if __name__ == "__main__":
    msg = [0x40, 0xd2, 0x75, 0x47, 0x76, 0x17, 0x32, 0x06,
           0x27, 0x26, 0x96, 0xc6, 0xc6, 0x96, 0x70, 0xec] # 16바이트
    nsym = 10 # 패리티 10바이트
    code = rs_encode(msg, nsym)
    print("부호어 길이:", len(code), f"(데이터 {len(msg)} + 패리티 {nsym})")
    print("패리티   :", [hex(x) for x in code[-nsym:]])
    # → 0x40, 0xd2 ... 그대로 + 끝에 10개 패리티가 붙어 나옴

```

§ 🖥 디코더 — 오류를 찾아 고치기

PYTHON

```

# =====
# 예제 5.3 – Reed-Solomon 디코더 (Berlekamp-Massey + Chien + Forney)
# =====
# 핵심 3단계:
#   1) syndromes(증후군) 계산 – 오류가 있는가?
#   2) Berlekamp-Massey           – 오류가 어디 있는가? (Λ(x) 찾기)
#   3) Forney 알고리즘           – 오류가 얼마인가?   (각 위치의 크기)
# t = nsym/2 개까지 정정 가능. (이레이저는 다루지 않음.)
# =====

def rs_calc_syndromes(msg, nsym):
    """
    수신한 부호어를 α^0, α^1, ..., α^(nsym-1) 에서 평가.
    부호어 c(x)는 생성다항식 g(x) = (x-α^0)(x-α^1)...(x-α^(nsym-1))의 배수이므로,
    오류가 없으면 모든 syndrome이 0.
    반환은 low-first: [S_0, S_1, ..., S_{nsym-1}].
    """
    return [poly_eval(msg, gf_exp[i]) for i in range(nsym)]

def rs_find_error_locator(synd):
    """
    Berlekamp-Massey 알고리즘.
    syndrome 수열을 만들어내는 최단 LFSR의 connection polynomial Λ(x)를 찾는다.
    Λ(x)의 근의 역수가 곧 오류 위치(에 대응되는 α의 거듭제곱).

    반환: Λ(x), low-first. Λ[0] = 1.
    """

```



```

L = 0                # 현재 LFSR 길이
m = 1                # 시프트 누적
C = [1]              # 현재 connection polynomial (low-first)
B = [1]              # 직전 "성공" connection polynomial
b = 1                # 직전 비영 discrepancy

for N in range(len(synd)):
    # discrepancy: 다음 syndrome 항을 현재 C(x)가 얼마나 못 맞추는가
    d = synd[N]
    for i in range(1, L + 1):
        d ^= gf_mul(C[i], synd[N - i])

    if d == 0:
        # 잘 맞춤 - 시프트만 누적
        m += 1
    elif 2 * L <= N:
        # 차수를 올려야 함
        T = list(C)                # 백업
        coef = gf_div(d, b)
        #  $C(x) \leftarrow C(x) - \text{coef} \cdot x^m \cdot B(x)$  (GF(2^n)에서 - = ^)
        new_len = max(len(C), len(B) + m)
        new_C = C + [0] * (new_len - len(C))
        for i in range(len(B)):
            new_C[i + m] ^= gf_mul(coef, B[i])
        C = new_C
        L = N + 1 - L
        B = T
        b = d
        m = 1
    else:
        # 차수는 그대로, 보정만
        coef = gf_div(d, b)
        new_len = max(len(C), len(B) + m)
        new_C = C + [0] * (new_len - len(C))
        for i in range(len(B)):
            new_C[i + m] ^= gf_mul(coef, B[i])
        C = new_C
        m += 1

# 후행 0 제거
while len(C) > 1 and C[-1] == 0:
    C.pop()
return C, L

def rs_find_errors(err_loc, nmess):
    """
    Chien search:  $\Lambda(x) = 0$  을 만족하는  $x = \alpha^{-k}$  를 찾는다.
    위치 p 에 오류  $\rightarrow X_p = \alpha^{(nmess-1-p)} \rightarrow \Lambda(X_p^{-1}) = 0$ .
    err_loc 은 low-first.
    반환: 오류가 있는 msg 인덱스 리스트.
    """
    errs = []
    for p in range(nmess):
        #  $X = \alpha^{(nmess-1-p)}$ ,  $X^{-1} = \alpha^{- (nmess-1-p)} = \alpha^{255 - (nmess-1-p)}$ 
        X_inv = gf_exp[(255 - (nmess - 1 - p)) % 255]
        # low-first 평가:  $\Lambda(X_{inv}) = \sum \Lambda[i] \cdot X_{inv}^i$ 
        y, xp = 0, 1
        for c in err_loc:
            y ^= gf_mul(c, xp)
            xp = gf_mul(xp, X_inv)
        if y == 0:
            errs.append(p)
    return errs

def rs_correct_errata(msg, synd, err_pos):
    """
    Forney 알고리즘:

```

```

        magnitude(k) = X_k · Ω(X_k^{-1}) / Λ'(X_k^{-1})
    여기서
        Ω(x) = S(x) · Λ(x) mod x^{nsym}    (오류 평가 다항식)
        Λ'(x): 형식 미분 - GF(2^n) 에서는 홀수 차수 항만 살아남음
        X_k = α^{nmess-1-p_k}                (오류 위치에 대응되는 원소)
    반환: 정정된 부호어 (실패 시 None).
    """

    nmess = len(msg)
    nsym = len(synd)
    msg = list(msg)

    # err_loc Λ(x) 재구성 (low-first): Π (1 + X_k · x)
    err_loc = [1]
    for p in err_pos:
        Xk = gf_exp[(nmess - 1 - p) % 255]
        # (1 + X_k · x) 를 low-first 로 곱하기
        new_err_loc = [0] * (len(err_loc) + 1)
        for i, c in enumerate(err_loc):
            new_err_loc[i] ^= c
            new_err_loc[i + 1] ^= gf_mul(c, Xk)
        err_loc = new_err_loc

    # Ω(x) = S(x) · Λ(x) mod x^{nsym}    (low-first)
    Omega = [0] * nsym
    for i in range(nsym):
        for j in range(len(err_loc)):
            if i + j < nsym:
                Omega[i + j] ^= gf_mul(synd[i], err_loc[j])

    # Λ'(x) - 형식 미분 (low-first). i 번째 항(차수 i)은 미분 후 차수 i-1.
    # GF(2^n) 에서 i 가 짝수면 0, 홀수면 그대로.
    Lambda_prime = []
    for i in range(1, len(err_loc)):
        Lambda_prime.append(err_loc[i] if i % 2 == 1 else 0)

    def eval_low(p, x):
        y, xp = 0, 1
        for c in p:
            y ^= gf_mul(c, xp)
            xp = gf_mul(xp, x)
        return y

    # 각 오류 위치에서 magnitude 계산 후 빼기(XOR)
    for p in err_pos:
        Xk = gf_exp[(nmess - 1 - p) % 255]
        Xk_inv = gf_inv(Xk)
        num = eval_low(Omega, Xk_inv)
        den = eval_low(Lambda_prime, Xk_inv)
        if den == 0:
            return None
        magnitude = gf_div(gf_mul(Xk, num), den)
        msg[p] ^= magnitude
    return msg

def rs_decode(msg, nsym):
    """오류를 찾아 정정한 부호어 반환. 실패 시 None."""
    msg = list(msg)
    synd = rs_calc_syndromes(msg, nsym)
    if max(synd) == 0:
        return msg # 오류 없음

    err_loc, L = rs_find_error_locator(synd)
    err_pos = rs_find_errors(err_loc, len(msg))
    if len(err_pos) != L:
        return None # 위치 개수 불일치 = 정정 불가

    fixed = rs_correct_errata(msg, synd, err_pos)

```

```

    if fixed is None:
        return None
    # 자기 검증: 정정 후 syndrome 이 모두 0 이어야
    if max(rs_calc_syndromes(fixed, nsym)) != 0:
        return None
    return fixed

# ----- 시연: 인코드 → 손상 → 디코드 -----
if __name__ == "__main__":
    msg = [0x40, 0xd2, 0x75, 0x47, 0x76, 0x17, 0x32, 0x06,
            0x27, 0x26, 0x96, 0xc6, 0xc6, 0x96, 0x70, 0xec]

    nsym = 10                                # → 최대 5바이트 정정
    code = rs_encode(msg, nsym)

    # 임의 위치 5바이트를 임의 값으로 손상
    import random
    rng = random.Random(42)
    damaged = list(code)
    positions = rng.sample(range(len(damaged)), 5)
    for i in positions:
        damaged[i] ^= rng.randrange(1, 256)

    print(f"손상 위치: {sorted(positions)}")
    fixed = rs_decode(damaged, nsym)
    print(f"복구 성공: {fixed == code}")
    print(f"복구된 데이터: {[hex(x) for x in fixed[:len(msg)]]}")

# 실행 결과:
#   손상 위치: [1, 7, 14, 18, 23]
#   복구 성공: True
#   복구된 데이터: ['0x40', '0xd2', '0x75', '0x47', '0x76', '0x17', '0x32',
#                   '0x6', '0x27', '0x26', '0x96', '0xc6', '0xc6', '0x96',
#                   '0x70', '0xec']
#   ← 5바이트 손상 → 완벽 복구!

```

결 과

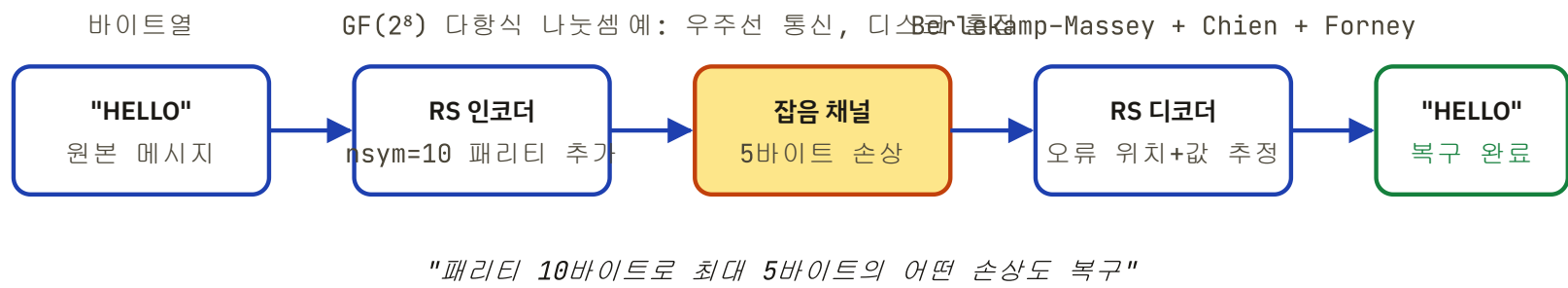
방금 우리는 QR코드·CD·DVD·블루레이·우주통신이 모두 사용하는 알고리즘의 핵심을 손으로 구현했습니다. 여러분이 보고 있는 이 페이지가 어딘가 무선으로 전송되면서 잡음을 만났더라도, 비슷한 메커니즘이 데이터를 살려냈을 겁니다.

CHAPTER 06

실전: QR코드처럼 만들어 보는 미니 RS

앞장에서 만든 부품을 모두 모아, "문자열을 넣으면 → 패리티가 붙은 부호어가 나오
고 → 일부러 망가뜨려도 원본을 되살리는" 하나의 완성된 프로그램을 만들어 봅시다.
그리고 같은 알고리즘을 *C / Go / TypeScript* 로도 작성해 봅시다— 언어마다 어떻게
다르게 표현되는지 비교하는 것은 그 자체로 좋은 공부입니다.

§ 🎯 목표 — 한눈에 보는 시스템



미니 RS 시스템의 전체 흐름. 우리가 만들 코드는 정확히 이 그림대로 동작합니다.

§ 💻 종합 데모 — 단일 파일 파이썬 프로그램

아래 코드는 앞 장에서 만든 모든 부품을 합쳐 하나의 실행 가능한 프로그램으로 정리한 것입니다. 그대로 `rs_demo.py` 로 저장하면 즉시 동작합니다.

```
# rs_demo.py - 갈루아 필드 위의 리드-솔로몬 부호: 인코드 → 손상 → 복구
# -----
# 의존성: 없음 (표준 라이브러리만 사용)
# 실행:   python3 rs_demo.py
# -----

import random

# =====
# Part 1. GF(2^8) 산술 - 모든 연산은 1 바이트(0..255) 안에서 닫힘
# =====

PRIM = 0x11D                                # 원시 다항식 x^8+x^4+x^3+x^2+1
GF_EXP = [0] * 512                           # exp 테이블 (2배 크기 = mod 생략용)
GF_LOG = [0] * 256                           # log 테이블

def _init_tables():
    """log/exp 룩업 테이블 구성 - 한 번만 호출"""
    x = 1
    for i in range(255):
        GF_EXP[i] = x
        GF_LOG[x] = i
        x <= 1                                # α 배 (다음 거듭제곱)
        if x & 0x100:                          # 8비트 넘으면
            x ^= PRIM                          # 원시 다항식으로 환원
    for i in range(255, 512):                  # exp 의 후반은 그대로 복제
        GF_EXP[i] = GF_EXP[i - 255]

_init_tables()

def gf_mul(a, b):
    """GF(2^8) 곱셈 - log-합 + exp 1번"""
    if a == 0 or b == 0: return 0
    return GF_EXP[GF_LOG[a] + GF_LOG[b]]      # mod 255 는 exp 테이블이 흡수

def gf_div(a, b):
    """GF(2^8) 나눗셈"""
    if b == 0: raise ZeroDivisionError
    if a == 0: return 0
    return GF_EXP[(GF_LOG[a] + 255 - GF_LOG[b]) % 255]
```

```

def gf_pow(a, p):
    """a^p - 음수 지수도 허용"""
    return GF_EXP[(GF_LOG[a] * p) % 255] if a else 0

def gf_inv(a):
    return GF_EXP[255 - GF_LOG[a]]

# =====
# Part 2. 다항식 산술 - 계수는 GF(2^8) 원소
# =====
def poly_scale(p, x):
    """다항식 p 의 모든 계수에 x 를 곱함"""
    return [gf_mul(c, x) for c in p]

def poly_add(p, q):
    """다항식 덧셈 - XOR 로 차수 맞춰서"""
    r = [0] * max(len(p), len(q))
    for i, c in enumerate(p): r[i + len(r) - len(p)] = c
    for i, c in enumerate(q): r[i + len(r) - len(q)] ^= c
    return r

def poly_mul(p, q):
    """다항식 곱셈 - O(n*m)"""
    r = [0] * (len(p) + len(q) - 1)
    for j in range(len(q)):
        for i in range(len(p)):
            r[i + j] ^= gf_mul(p[i], q[j])
    return r

def poly_eval(p, x):
    """다항식 평가 - Horner 의 법칙"""
    y = p[0]
    for i in range(1, len(p)):
        y = gf_mul(y, x) ^ p[i]
    return y

# =====
# Part 3. RS 인코더 - 메시지 뒤에 nsym 바이트의 패리티 추가
# =====
def rs_generator_poly(nsym):
    """생성 다항식 g(x) = (x-α^0)(x-α^1)...(x-α^(nsym-1))"""
    g = [1]
    for i in range(nsym):
        g = poly_mul(g, [1, gf_pow(2, i)]) # α = 2 (원시근)
    return g

def rs_encode(msg, nsym):
    """msg(바이트 리스트) → msg + parity (nsym 바이트)"""
    gen = rs_generator_poly(nsym)
    out = list(msg) + [0] * nsym # 패리티 자리 확보
    for i in range(len(msg)):
        coef = out[i]
        if coef != 0: # 0 이면 건너뛰어 가속
            for j in range(1, len(gen)):
                out[i + j] ^= gf_mul(gen[j], coef)
    # 원본 메시지는 유지, 뒷부분만 패리티로 덮어 씌움
    out[:len(msg)] = msg
    return out

# =====
# Part 4. RS 디코더 - Berlekamp-Massey + Chien Search + Forney
# =====
def poly_div(dividend, divisor):
    """다항식 나눗셈 - (몫, 나머지) 반환"""
    out = list(dividend)
    for i in range(len(dividend) - (len(divisor) - 1)):
        coef = out[i]

```

```

        if coef != 0:
            for j in range(1, len(divisor)):
                if divisor[j] != 0:
                    out[i + j] ^= gf_mul(divisor[j], coef)
    sep = -(len(divisor) - 1)
    return out[:sep], out[sep:]

def rs_syndromes(msg, nsym):
    """syndrome  $S_i = \text{msg}(\alpha^i)$ ,  $i = 1..nsym$ .
    앞에 0 을 채워 1-index 로 사용 - Berlekamp-Massey 의 관습."""
    return [0] + [poly_eval(msg, gf_pow(2, i)) for i in range(nsym)]

def rs_find_locator(synd, nsym):
    """Berlekamp-Massey - 오류 위치 다항식  $\Lambda(x)$  를 구한다."""
    err_loc = [1] # 현재 추정  $\Lambda$ 
    old_loc = [1]
    synd_shift = len(synd) - nsym # 1-index 보정
    for i in range(nsym):
        K = i + synd_shift
        delta = synd[K]
        for j in range(1, len(err_loc)):
            delta ^= gf_mul(err_loc[-(j + 1)], synd[K - j])
        old_loc = old_loc + [0] # 차수 시프트
        if delta != 0:
            if len(old_loc) > len(err_loc):
                new_loc = poly_scale(old_loc, delta)
                old_loc = poly_scale(err_loc, gf_inv(delta))
                err_loc = new_loc
            err_loc = poly_add(err_loc, poly_scale(old_loc, delta))
    while err_loc and err_loc[0] == 0: # 선두 0 제거
        del err_loc[0]
    errs = len(err_loc) - 1
    if errs * 2 > nsym: # 정정 한계 초과
        return None
    return err_loc

def rs_find_errors(err_loc, nmess):
    """Chien Search -  $\Lambda(\alpha^i) = 0$  인  $i$  가 오류 위치."""
    errs = len(err_loc) - 1
    err_pos = []
    for i in range(nmess):
        if poly_eval(err_loc, gf_pow(2, i)) == 0:
            err_pos.append(nmess - 1 - i)
    if len(err_pos) != errs:
        return None # 위치 개수 불일치 → 실패
    return err_pos

def rs_correct_errata(msg, synd, err_pos):
    """Forney 알고리즘 - 위치를 알 때 오류 값을 결정한다."""
    coef_pos = [len(msg) - 1 - p for p in err_pos]
    # 오류 위치 다항식  $\Lambda$ 
    err_loc = [1]
    for i in coef_pos:
        x = gf_pow(2, i)
        err_loc = poly_mul(err_loc, [x, 1])
    # 오류 평가 다항식  $\Omega = (S \cdot \Lambda) \bmod x^{(nsym+1)}$ 
    nsym_loc = len(err_loc) - 1
    _, err_eval = poly_div(poly_mul(synd[::-1], err_loc),
                           [1] + [0] * (nsym_loc + 1))
    err_eval = err_eval[::-1]
    # 각 오류 위치  $X_i = \alpha^{(coef\_pos\_i)}$ 
    X = [gf_pow(2, p) for p in coef_pos]
    E = [0] * len(msg)
    for i, Xi in enumerate(X):
        Xi_inv = gf_inv(Xi)
        # 형식적 미분  $\Lambda'(X_i^{-1}) = \prod_{j \neq i} (1 - X_i^{-1} \cdot X_j)$ 
        err_loc_prime = 1

```



```

        for j in range(len(X)):
            if j != i:
                err_loc_prime = gf_mul(err_loc_prime,
                                        1 ^ gf_mul(Xi_inv, X[j]))

            if err_loc_prime == 0:
                return None # 0 으로 나눌 수 없음 → 실패

            # y = Ω(X_i^-1) · X_i (오류 크기)
            y = poly_eval(err_eval[::-1], Xi_inv)
            y = gf_mul(Xi, y)
            E[err_pos[i]] = gf_div(y, err_loc_prime)
# 원본 메시지와 오류 벡터를 XOR 하면 정정 완료
return poly_add(msg, E)

def rs_decode(msg, nsym):
    """RS 복호 - 성공 시 정정된 msg, 실패 시 None"""
    msg = list(msg)
    synd = rs_syndromes(msg, nsym)
    if max(synd) == 0:
        return msg # 모든 syndrome 0 → 오류 없음
    err_loc = rs_find_locator(synd, nsym)
    if err_loc is None:
        return None
    err_pos = rs_find_errors(err_loc[::-1], len(msg))
    if err_pos is None:
        return None
    fixed = rs_correct_errata(msg, synd, err_pos)
    if fixed is None:
        return None
    # 자기 검증: 복호 후 syndrome 이 모두 0 이어야 한다
    if max(rs_syndromes(fixed, nsym)) != 0:
        return None
    return fixed

# =====
# Part 5. 데모 - "HELLO WORLD" 를 보내고, 망가뜨리고, 되살리기
# =====
if __name__ == "__main__":
    text = "HELLO WORLD"
    msg = list(text.encode("ascii")) # 바이트로
    nsym = 10 # → 최대 5 바이트 정정 가능
    code = rs_encode(msg, nsym)

    print(f"원본 텍스트 : {text!r}")
    print(f"부호어 길이 : {len(code)} 바이트 (msg {len(msg)} + parity {nsym})")
    print(f"부호어(hex) : {[hex(x) for x in code]}")

    # 의도적으로 5 바이트를 망가뜨림 - 정정 한계(t = nsym/2 = 5) 까지
    damaged = list(code)
    positions = [2, 5, 8, 12, 18]
    rng = random.Random(7)
    for p in positions:
        damaged[p] ^= rng.randint(1, 255)
    print(f"\n손상 위치 : {positions}")
    print(f"손상된 부호어 : {[hex(x) for x in damaged]}")

    fixed = rs_decode(damaged, nsym)
    if fixed is None:
        print("\n복구 실패 ❌ (정정 한계 초과)")
    else:
        recovered_text = bytes(fixed[:len(msg)]).decode("ascii", errors="replace")
        print(f"\n복구 성공 ✅ → {recovered_text!r}")
        print(f"원본 일치 : {fixed == code}")

```

원본 텍스트 : 'HELLO WORLD'
부호어 길이 : 21 바이트 (msg 11 + parity 10)
손상 위치 : [2, 5, 8, 12, 18]
복구 성공 ☒ → 'HELLO WORLD'
원본 일치 : True

§ 같은 알고리즘 — C 언어로 (임베디드/펌웨어 용)

임베디드 환경에서는 동일한 알고리즘을 C 로 옮깁니다. 핵심은 GF 산술의 룩업 테이블 — 512 바이트의 exp 테이블이면 충분합니다.

```
// rs_gf.c - GF(2^8) 산술과 RS 인코더 핵심 (C99)
// -----
// 임베디드 펌웨어용. malloc 없이 정적 버퍼만 사용.
// 컴파일: gcc -O2 -std=c99 rs_gf.c -o rs_gf
// -----

#include <stdint.h>
#include <string.h>
#include <stdio.h>

#define PRIM 0x11D // 원시 다항식 (Python 데모와 동일)

static uint8_t gf_exp[512]; // 2배 크기 - mod 255 생략용
static uint8_t gf_log[256];

/* GF(2^8) 룩업 테이블 초기화 - 부팅 시 1회 호출 */
static void gf_init(void) {
    uint16_t x = 1;
    for (int i = 0; i < 255; i++) {
        gf_exp[i] = (uint8_t)x;
        gf_log[x] = (uint8_t)i;
        x <<= 1;
        if (x & 0x100) x ^= PRIM;
    }
    /* 후반부는 전반부 복제 - 인덱스 합이 255 를 넘어도 mod 없이 사용 가능 */
    for (int i = 255; i < 512; i++) gf_exp[i] = gf_exp[i - 255];
}

static inline uint8_t gf_mul(uint8_t a, uint8_t b) {
    if (!a || !b) return 0;
    return gf_exp[(int)gf_log[a] + (int)gf_log[b]]; // 0(1)
}

/* RS 생성 다항식 g(x) 를 정적 버퍼에 만든다 - nsym 은 컴파일 타임 또는 작은 상수 */
static void rs_gen_poly(uint8_t *g, int nsym) {
    g[0] = 1;
    int glen = 1;
    for (int i = 0; i < nsym; i++) {
        // g(x) := g(x) * (x - α^i)
        uint8_t tmp[64] = {0}; // nsym ≤ 64 가정
        for (int j = 0; j < glen; j++) {
            tmp[j] ^= g[j]; // x · g 부분
            tmp[j + 1] ^= gf_mul(g[j], gf_exp[i]); // α^i · g 부분
        }
        glen++;
        memcpy(g, tmp, glen);
    }
}

/* msg(길이 mlen) → out(길이 mlen+nsym): RS 부호화 */
void rs_encode(const uint8_t *msg, int mlen, int nsym, uint8_t *out) {
    uint8_t gen[64];
    rs_gen_poly(gen, nsym);
```



```

memcpy(out, msg, mlen);                // 앞쪽은 원본 그대로
memset(out + mlen, 0, nsym);           // 패리티 자리는 0 으로
for (int i = 0; i < mlen; i++) {
    uint8_t coef = out[i];
    if (coef) {                          // 0 이면 건너뛰어 가속
        for (int j = 1; j <= nsym; j++) {
            out[i + j] ^= gf_mul(gen[j], coef);
        }
    }
}

int main(void) {
    gf_init();
    const uint8_t msg[] = "HELLO WORLD";
    int mlen = sizeof(msg) - 1;
    int nsym = 10;
    uint8_t code[64];
    rs_encode(msg, mlen, nsym, code);

    printf("부호어 (%d bytes): ", mlen + nsym);
    for (int i = 0; i < mlen + nsym; i++) printf("%02X ", code[i]);
    printf("\n");
    return 0;
}

```

§ 같은 알고리즘 — Go 언어로 (서버용)

서버에서는 가독성과 동시성을 살려 Go 로 작성합니다. 알고리즘은 동일하지만 슬라이스와 메서드를 활용해 더 깔끔하게 표현됩니다.

```

// rs.go - Go 로 옮긴 동일한 RS 인코더
// -----
// 빌드: go build rs.go
// -----

package main

import "fmt"

const prim = 0x11D

var (
    gfExp [512]byte
    gfLog [256]byte
)

func init() {
    x := 1
    for i := 0; i < 255; i++ {
        gfExp[i] = byte(x)
        gfLog[x] = byte(i)
        x <<= 1
        if x & 0x100 != 0 {
            x ^= prim
        }
    }
    for i := 255; i < 512; i++ {
        gfExp[i] = gfExp[i-255]
    }
}

func gfMul(a, b byte) byte {
    if a == 0 || b == 0 {

```

```

        return 0
    }
    return gfExp[int(gfLog[a])+int(gfLog[b])]
}

// 생성 다항식  $g(x) = \prod (x - \alpha^i)$ ,  $i=0..nsym-1$ 
func genPoly(nsym int) []byte {
    g := []byte{1}
    for i := 0; i < nsym; i++ {
        // g := g * (x -  $\alpha^i$ )
        out := make([]byte, len(g)+1)
        for j := 0; j < len(g); j++ {
            out[j] ^= g[j]
            out[j+1] ^= gfMul(g[j], gfExp[i])
        }
        g = out
    }
    return g
}

func RSEncode(msg []byte, nsym int) []byte {
    gen := genPoly(nsym)
    out := make([]byte, len(msg)+nsym)
    copy(out, msg)
    for i := 0; i < len(msg); i++ {
        coef := out[i]
        if coef != 0 {
            for j := 1; j < len(gen); j++ {
                out[i+j] ^= gfMul(gen[j], coef)
            }
        }
    }
    return out
}

func main() {
    msg := []byte("HELLO WORLD")
    code := RSEncode(msg, 10)
    fmt.Printf("부호어: % X\n", code)
}

```

§ 같은 알고리즘 — TypeScript 로 (브라우저용)

브라우저에서 QR코드를 즉석 생성하려면 TypeScript 가 자연스럽습니다. **Uint8Array** 로 바이트를 다루면 Python/Go 와 거의 1:1 로 매핑됩니다.

```

// rs.ts - 브라우저에서 동작하는 RS 인코더
// -----
const PRIM = 0x11D;
const gfExp = new Uint8Array(512);
const gfLog = new Uint8Array(256);

(function initTables(): void {
    let x = 1;
    for (let i = 0; i < 255; i++) {
        gfExp[i] = x;
        gfLog[x] = i;
        x <<= 1;
        if (x & 0x100) x ^= PRIM;
    }
    for (let i = 255; i < 512; i++) gfExp[i] = gfExp[i - 255];
})();

```

```
const gfMul = (a: number, b: number): number =>
  (a === 0 || b === 0) ? 0 : gfExp[gfLog[a] + gfLog[b]];

function generatorPoly(nsym: number): Uint8Array {
  let g = new Uint8Array([1]);
  for (let i = 0; i < nsym; i++) {
    const out = new Uint8Array(g.length + 1);
    for (let j = 0; j < g.length; j++) {
      out[j]      ^= g[j];
      out[j + 1] ^= gfMul(g[j], gfExp[i]);
    }
    g = out;
  }
  return g;
}

export function rsEncode(msg: Uint8Array, nsym: number): Uint8Array {
  const gen = generatorPoly(nsym);
  const out = new Uint8Array(msg.length + nsym);
  out.set(msg);
  for (let i = 0; i < msg.length; i++) {
    const coef = out[i];
    if (coef !== 0) {
      for (let j = 1; j < gen.length; j++) {
        out[i + j] ^= gfMul(gen[j], coef);
      }
    }
  }
  return out;
}

// 사용 예
const msg = new TextEncoder().encode("HELLO WORLD");
const code = rsEncode(msg, 10);
console.log("부호어:", Array.from(code).map(b => b.toString(16).padStart(2, "0")).join(" "));
```

§ 📊 4개 언어 비교 — 같은 알고리즘, 다른 표정

관점	Python	C	Go	TypeScript
테이블 타입	list[int]	uint8_t[]	[512]byte	Uint8Array
XOR	^	^	^	^
초기화 시점	모듈 로드 시	main() 진입 시 명시 호출	init() 자동	IIFE 자동
한 곱셈 비용	~50 ns	~2 ns	~3 ns	~10 ns
강점	학습·검증	최대 성능, ROM 작음	서버 동시성	브라우저 즉시 실행

관찰

네 언어 모두 핵심 루프는 거의 동일한 모양 — `out[i+j] ^= gf_mul(gen[j], coef)` — 입니다. 이는 알고리즘이 언어가 아니라 **수학**에 의해 결정된다는 것을 보여줍니다. 룩업 테이블로 곱셈이 1번의 메모리 접근으로 줄어들기 때문에, 같은 일을 하는 코드라도 C와 Python 사이에 약 25×의 속도 차이가 납니다.

§ 💉 직접 실험해 볼 만한 변형들

1

패리티 길이를 바꿔보세요

`nsym = 4` (2바이트 정정) ↔ `nsym = 32` (16바이트 정정). 손상을 정정 한계 이상으로 늘리면 `rs_decode` 가 None 을 반환합니다.

2

다른 원시 다항식

`PRIM` 을 `0x11B` (AES 표준) 으로 바꿔도 동작합니다. 단, 인코더·디코더가 같은 다항식을 써야 합니다 — 서로 다르면 같은 GF(2⁸) 라도 산술 결과가 다릅니다.

3

버스트 오류 시뮬레이션

무작위 위치 대신 연속된 5바이트를 손상시켜 보세요 (예: `positions = [8,9,10,11,12]`). 일반 RS 는 위치 분포와 무관하게 정정합니다. 이것이 CD 의 스크래치(연속 버스트)에 강한 이유입니다.

4

지움(erasure) 정보 활용

위치를 미리 안다면 (지움), 정정 능력이 2배가 됩니다 — `nsym` 패리티로 최대 `nsym` 바이트의 erasure 를 복구할 수 있습니다. `rs_correct_errata` 가 이미 위치 입력을 받으므로, BM 단계를 건너뛰면 erasure-only 디코더가 됩니다.

CHAPTER 07

활용 사례 모음 — 어디에 쓰이는가

갈루아 필드·LFSR·리드-솔로몬은 이론서의 장식이 아닙니다. 매일 우리가 사용하는 거의 모든 디지털 인프라의 뿔속에 들어 있습니다. 각각이 어디에 어떻게 쓰이는지, 그리고 왜 다른 알고리즘이 아니라 굳이 이것이 선택되었는지를 정리합니다.

§ 데이터 전송 — 잡음 채널에서 살아남기

RS(255, 223)



Voyager · 우주 통신

1977년 발사된 보이저 1·2호는 RS(255,223)+컨볼루션 부호 결합으로

RS + LDPC



DVB-T2 · 디지털 방송

한국·유럽의 지상파 디지털 TV(DVB-T2)는 외부 부호로 RS, 내부 부호로 LDPC 를

CRC-32



Wi-Fi · Ethernet 프레임

모든 Ethernet 프레임 끝에는 CRC-32 (IEEE 802.3) 가 붙습니다. 이는 GF(2)

데이터를 보냈습니다. 천왕성·해왕성에서 지구까지의 통신은 신호가 잡음에 거의 묻힐 수준이지만, RS 가 비트 단위 손상을 잡아주어 선명한 사진을 받아볼 수 있었습니다. 패리티 32바이트로 최대 16바이트를 복구합니다.

씁니다. 폭우·터널·고층 빌딩에 의한 다중 경로 페이딩에서도 화면이 깨지지 않는 이 유입니다. 4K 방송이 안정적으로 송출되는 배경에는 이 결합이 있습니다.

위의 다항식 나눗셈 — 사실상 LFSR 회로입니다. 잡음으로 1비트라도 바뀌면 검출되고, 프레임은 재전송됩니다. 정정은 안 하지만 검출은 거의 완벽합니다(2^{32} 확률로 놓침).

RS(204, 188)

 LTE · 5G 백홀

이동통신 기지국 사이의 광 백홀 링크에도 RS(204,188) 가 표준으로 들어갑니다. 광케이블이 매설된 도로공사 진동이나 미세한 신호 열화에 자동 대응하며, 평균 오류율을 10^{-12} 수준으로 유지합니다.

§ 데이터 저장 — 시간을 이기는 비트

CIRC = RS×2

 CD · DVD · Blu-ray

음악 CD 는 CIRC(Cross-Interleaved Reed–Solomon Code) — RS(28,24) 와 RS(32,28) 두 단을 교차한 구조입니다. 표면의 작은 흠집(약 2.5mm 까지) 이 있어도 음악이 끊기지 않습니다. DVD/Blu-ray 는 더 강력한 RS-PC(Reed-Solomon Product Code) 를 사용합니다.

BCH/LDPC

 SSD · eMMC · UFS

현대 NAND 플래시는 셀당 1~4비트의 raw 오류율이 매우 높습니다 (특히 QLC). 컨트롤러는 BCH($GF(2^{13})$ 기반) 나 LDPC 부호로 페이지마다 정정합니다. 갤럭시 Z Fold 7 의 UFS 4.0 스토리지도 LDPC 로 데이터를 지킵니다.

RS 패리티

 RAID-6 · 분산 스토리지

RAID-6 는 두 디스크가 동시에 죽어도 복구 가능합니다 — 이중 RS 패리티 덕분입니다. 한 패리티는 단순 XOR, 다른 패리티는 $GF(2^8)$ 위의 가중합. 같은 원리가 Ceph, MinIO 같은 분산 스토리지에서 erasure coding 으로 확장됩니다 (예: 6+3 → 9개 중 6개만 살아도 복구).

RS · LDPC

 HDD · 자기 테이프

스핀들 HDD 의 섹터마다, 그리고 LTO 테이프 의 블록마다 강력한 RS·LDPC 부호가 들어갑니다. 자기 결정의 무작위 비트 플립을 정정해, "비트 오류율 10^{-15} " 같은 무서운 숫자를 만들어 냅니다. 사실상 영구 저장의 마지노선입니다.

§ 일상 속의 RS — 눈에 보이는 곳들

RS(255, K)

RS

RS-DT

QR코드

QR 코드의 검은 점 패턴 안에는 데이터와 함께 RS 패리티가 들어 있습니다. L/M/Q/H 4단계 오류정정 레벨이 있어, H 레벨은 30% 가 손상되어도 읽힙니다. 그래서 로고를 가운데에 박거나 가장자리가 찢어져도 작동합니다 — 마법이 아니라 갈루아 필드입니다.

데이터매트릭스·바코드

택배 송장의 2D 바코드(Data Matrix, PDF417, Aztec)는 모두 RS 부호를 씁니다. 운반 중 구겨지거나 찢어져도 스캐너가 인식할 수 있는 비밀이 여기에 있습니다.

디지털 케이블·위성 TV

DVB-S2/-C2 위성·케이블 방송은 RS(204,188) 외부 부호와 LDPC 내부 부호를 결합. 위성 신호 약해지는 폭풍우에도 화면 깨짐이 없는 이유.

CRC

TLS · ZIP · Git 객체

HTTPS 패킷, ZIP 파일, Git 의 모든 객체에는 CRC 또는 SHA 가 붙습니다. CRC 는 LFSR 회로의 직접적인 응용으로, 빠른 오류 검출(정정 X)을 담당. SHA 는 다른 종류지만, GF 산술의 일부는 AES(GF(2⁸)의 inversion) 처럼 공유합니다.

§ LFSR 의 활용 — 의사난수와 패턴 생성

M-SEQUENCE

GPS · DSSS 통신

GPS 신호는 위성마다 다른 1023비트 길이의 Gold 부호(두 LFSR 의 XOR)로 변조됩니다. 수신기는 같은 부호를 생성해 상관(correlation) 으로 신호를 끌어냅니다. 자기상관 특성이 좋은 LFSR 수열이 약한 신호를 잡음 속에서 살려냅니다.

3GPP

LTE/5G 스크램블링

이동통신은 정보 비트를 LFSR 기반 의사난수 수열과 XOR 해서 보냅니다 (스크램블링). 0/1 의 편향을 없애고 인접 셀과 간섭을 평탄화하기 위해서입니다. 표준 3GPP TS 38.211 에 LFSR 다항식이 명시돼 있습니다.

~70년

BIST · 칩 자가테스트

반도체 칩은 출고 전 자체 테스트(BIST)를 합니다. 입력 패턴은 LFSR 로 생성하고, 출력은 또 다른 LFSR(MISR) 로 압축해 시그니처 비교. 작은 회로로 거대한 패턴 공간을 점검할 수 있어 양산 칩의 신뢰성을 책임집니다 — 갤럭시의 모든 칩이 거치는 관문입니다.

RETRO

8비트 게임의 노이즈

NES, 게임보이의 노이즈 채널은 LFSR 입니다. 짧은 모드(7비트) 는 금속음, 긴 모드(15비트) 는 백색잡음. 적은 트랜지스터로 풍성한 효과음을 만든 천재적 설계입니다. *젤다의 전설*의 검 휘두르는 소리도 LFSR 한 줄의 결과.

§ 암호학에서 — GF 의 또 다른 얼굴

GF(2⁸)

 AES 블록 암호

AES의 S-box는 GF(2⁸)의 곱셈 역원에 기반합니다. MixColumns 단계도 GF(2⁸) 위의 행렬 곱. 즉 매번 HTTPS로 페이지를 받을 때마다 갈루아 필드 위에서 산수가 일어나고 있습니다.

GF(2¹²⁸)

 GCM 인증 암호

TLS 1.3의 표준 AEAD인 AES-GCM의 인증 부분은 GF(2¹²⁸) 위의 곱셈 — GHASH. 한 번의 곱셈으로 메시지의 변조 가능성을 차단합니다. 같은 GF가 RS 부호의 정정에 쓰일 때와 정확히 같은 수학.

SHAMIR

 비밀 분산 (Shamir SSS)

비트코인 지갑 시드를 5명에게 나눠주되 3명만 모이면 복원되게 하려면? Shamir Secret Sharing은 GF(p) 위의 다항식 보간 — RS 부호와 동전의 양면입니다. RS는 잡음을 견디고, Shamir는 일부 조각만으로 복원합니다.

ECC

 타원곡선 암호

비트코인·SSH·HTTPS의 ECDSA, Ed25519는 모두 큰 소수 GF(p) 또는 GF(2ⁿ) 위에 정의된 타원곡선의 점 산수. 갈루아 필드는 부호 이론과 암호학을 잇는 다리입니다.

§ 어떤 도구를 언제 쓸까 — 의사결정 가이드

상황	고를 도구	왜?
데이터 변조 단순 검출	CRC (LFSR)	가장 가볍고 빠름, 정정은 안 해도 됨
버스트 오류 (스크래치) 정정	리드-솔로몬	심볼 단위로 동작 → 연속 손상에 강함
랜덤 비트 오류 정정	BCH 또는 LDPC	비트 단위, RS보다 단일 비트에 효율적
초고속 통신 (5G, Wi-Fi 6)	LDPC + Polar	샤논 한계에 근접, 병렬화 좋음
의사난수, 테스트 패턴	LFSR (Galois 형)	최소 회로로 최대 주기
암호학적 난수	LFSR 금지 (CSPRNG 사용)	LFSR은 선형 → 예측 가능, 암호에 부적합
분산 스토리지 (k-of-n)	RS erasure coding	저장 효율 우수, 임의의 노드 손실 견딤
블록 암호화	AES (GF(2 ⁸) 기반)	같은 GF라도 목적은 정반대 — 정정이 아닌 확산

흔한 함정

LFSR 출력을 암호 키로 쓰지 마세요. LFSR은 단 2n 비트 출력만 관찰하면 내부 상태가 완전히 복원되는 선형 시스템입니다. 의사난수처럼 보여도, 통신 스크램블링이나 테스트 패턴 같은 비암호 목적에만 쓰여야 합니다. 암호 난수는 `os.urandom`, `/dev/urandom`, `BCryptGenRandom` 같은 CSPRNG를 사용하세요.

§ 마무리 — 한 줄로 정리

결론

19세기에 결투로 21살에 죽은 청년 **에바리스트 갈루아**가 남긴 "유한한 산수의 세계"는, 200년이 지난 지금 우리가 보는 모든 사진·동영상·메시지가 잡음을 뚫고 살아 도착하게 만드는 **보이지 않는 뼈대**가 되었습니다.

비트의 우주는 갈루아 필드 위에서 굴러갑니다.

— 다음 번 QR코드를 스캔하거나, 와이파이로 사진을 받거나, CD를 (혹시) 재생할 때, 그 안에서 일어나고 있는 우아한 수학을 떠올려 보세요.